Internet das coisas

Segurança no Desenvolvimento de Software

Segurança nos protocolos de comunicação

Prof. Ana Beatriz Morais

OUT/2026

SENAI
PELO FUTURO DO TRABALHO

O que veremos hoje

- **Diferenciar TCP e UDP.**
- **Identificar riscos de interceptação, falsificação e indisponibilidade.**
- **Escolher mecanismos de proteção adequados.**
- **Relacionar segurança de rede com desenvolvimento de software.**
- **Atividades**



Protocolos de comunicação TCP e UDP

Modelo OSI

As 7 camadas do Modelo OSI (Open Systems Interconnection), desenvolvido pela ISO, organizam a comunicação de rede em etapas lógicas, da mais próxima do usuário até o hardware físico.



Protocolos de comunicação TCP e UDP

- A **principal diferença** é:
  - o **TCP organiza e acompanha a entrega dos dados;**
  - o **UDP envia os dados sem garantir que eles chegarão** ou **em qual ordem**.

TCP vs UDP

Característica	TCP	UDP
Conexão	Orientado à conexão (handshake)	Sem conexão
Confirmação de Entrega	Sim, com verificação de erro	Não há confirmarção
Velocidade (Speed)	Mais lenta (controle rigoroso)	Mais rápida (sem controle)
Integridade dos Dados	Garantida	Não garantida
Exemplos de Protocolos	HTTP, HTTPS, SMTP, FTP	DNS, DHCP, SNMP, VoIP

DLTEC.com.br

Protocolos de comunicação TCP e UDP

- TCP
  - Exemplo:
    - O envio de um documento completo.
    - Você precisa enviar um documento dividido em várias partes e todas precisam chegar e ser colocadas na ordem correta.
    - O que o TCP faz?
      - Estabelece uma conexão antes de transmitir os dados.
      - Confirma o recebimento dos dados.
      - Retransmite dados perdidos.
      - Entrega os dados em ordem para a aplicação.
    - Caso de uso:
      - Navegação web (HTTPS), e-mails, mensagens instantâneas e APIs em geral.

Protocolos de comunicação TCP e UDP

- UDP
  - Exemplo:
    - Em uma chamada de voz, o áudio precisa chegar no momento certo. Se um pequeno trecho se perder, esperar por ele pode ser pior do que continuar a conversa.
    - O que o UDP faz?
      - Envia dados sem estabelecer uma conexão previamente no próprio protocolo.
      - Não confirma o recebimento.
      - Não retransmite automaticamente dados perdidos.
      - Não garante a ordem de chegada.
    - Caso de uso:
      - Chamadas de voz e vídeo (VoIP), streaming em tempo real, jogos online e redes VPN/IoT sensíveis à latência

Protocolos de comunicação TCP e UDP

- TCP ser chamado de “confiável” significa que ele cuida da entrega dos dados.
- Isso não significa que os dados estejam criptografados.
- TCP e UDP precisam de mecanismos adicionais de segurança, como TLS e DTLS, respectivamente.
    - O que é TLS?
        - Transport Layer Security é um protocolo de segurança criptográfica projetado para proteger a comunicação em redes de computadores, como a internet, garantindo confidencialidade, integridade e autenticação de dados. Funciona sobre o TCP, exigindo uma conexão orientada a fluxo confiável e ordenada.
    - O que é DTLS?
        - Datagram Transport Layer Security é uma adaptação direta do TLS feita para proteger aplicações baseadas em UDP (User Datagram Protocol) em vez de TCP. Funciona sobre o UDP, sendo projetado para redes não confiáveis onde pacotes podem se perder ou chegar fora de ordem.

Identificar riscos de interceptação, falsificação e indisponibilidade

O que precisamos proteger?

- A segurança da comunicação envolve proteger o conteúdo transmitido, verificar quem participa da troca de informações e manter os serviços funcionando. Para isso, trabalhamos com quatro objetivos.
  - **1. Confidencialidade**
    - **Significa permitir que somente pessoas ou sistemas autorizados tenham acesso às informações.**
    - **Quando um usuário envia uma senha, um documento ou um dado pessoal, essas informações não devem ficar legíveis para qualquer pessoa que consiga observar o tráfego da rede.**
    - Exemplo: um aluno acessa o portal da escola e informa usuário e senha. A comunicação deve ser criptografada para evitar a leitura desses dados durante o transporte.
    - Controles relacionados:
      - Criptografia da comunicação, como HTTPS.
      - Controle de acesso às informações.
      - Proteção de credenciais e chaves criptográficas.

O que precisamos proteger?

- **2. Integridade**
  - **Significa proteger as informações contra alterações indevidas e permitir a detecção dessas alterações.**
  - **Não basta esconder uma informação: também precisamos garantir que ela não seja modificada silenciosamente durante a comunicação.**
  - Exemplo: uma requisição informa uma transferência de R$ 100. Se alguém tentar alterar o valor para R$ 1.000 durante o transporte, a proteção de integridade deve permitir detectar a adulteração.
  - Controles relacionados:
    - Mecanismos criptográficos de integridade, como os utilizados pelo TLS.
    - Assinaturas digitais, quando adequadas ao contexto.
    - Códigos de autenticação de mensagens, como HMAC.
  - Um hash enviado junto com a mensagem, sem proteção adicional, não basta contra um atacante que consiga substituir tanto a mensagem quanto o hash.

O que precisamos proteger?

- **3. Autenticidade**
  - **Significa verificar se uma pessoa, sistema ou mensagem realmente corresponde à identidade declarada.**
  - **Exemplo: antes de enviar a senha, o navegador precisa verificar se está se comunicando com o servidor correspondente ao domínio acessado.**
  - Também existe a autenticação do usuário: a API precisa verificar quem está tentando acessar os recursos.
  - Controles relacionados:
    - Certificados digitais.
    - Senhas e autenticação multifator.
    - Tokens de autenticação devidamente validados.
  - Diferença importante:
    - Autenticação: quem é você?
    - Autorização: o que você pode fazer?
  - Um aluno pode estar autenticado no sistema e, mesmo assim, não ter autorização para alterar notas.

O que precisamos proteger?

- **4. Disponibilidade**
  - **Significa manter os serviços acessíveis aos usuários autorizados quando forem necessários.**
  - Exemplo: o portal da escola precisa continuar funcionando durante o período de matrícula, mesmo com muitos acessos.
  - Controles relacionados:
    - Limites de requisições e conexões.
    - Timeouts.
    - Monitoramento.
    - Redundância e capacidade adequada.
    - Proteção contra ataques de negação de serviço.

O que precisamos proteger?

OBJETIVO	PERGUNTA	EXEMPLO
CONFIDENCIALIDADE	Alguém pode ler os dados no caminho?	Criptografia com HTTPS
INTEGRIDADE	Os dados podem ser alterados sem detecção?	Proteção criptográfica do TLS
AUTENTICIDADE	Estou falando com o servidor correto? Quem é o usuário?	Certificado e autenticação
DISPONIBILIDADE	O serviço continua atendendo?	Limites, timeouts e infraestrutura adequada

Interceptação de tráfego

- O que é sniffing?
  - Sniffing é a captura e análise de pacotes que trafegam por uma rede.
- Ferramentas de análise de tráfego, como o Wireshark, permitem observar características da comunicação, como:
  - Endereços IP.
  - Portas de origem e destino.
  - Protocolos utilizados.
  - Tamanho e horário dos pacotes.
  - Conteúdo transmitido, quando estiver disponível sem criptografia.

Interceptação de tráfego

- Uso legítimo
  - Administradores e desenvolvedores podem analisar tráfego para:
    - Investigar lentidão.
    - Identificar falhas de conexão.
    - Verificar retransmissões.
    - Entender a comunicação entre cliente e servidor.
    - Diagnosticar problemas de configuração.

Interceptação de tráfego

- Uso malicioso
  - Uma pessoa que consiga observar o tráfego pode tentar coletar:
    - Senhas.
    - Tokens de acesso.
    - Dados pessoais.
    - Mensagens.
    - Informações internas de sistemas.
- Estar na mesma rede não significa, automaticamente, conseguir visualizar todo o tráfego dos outros dispositivos. Isso depende da infraestrutura, do posicionamento e do acesso do observador.

Interceptação de tráfego

- Exemplo de comunicação sem criptografia
  - Imagine que um formulário envie esta requisição usando HTTP.

```
POST /login HTTP/1.1
Host: sistema.exemplo
Content-Type: application/x-www-form-urlencoded

usuario=aluno&senha=SenhaDeTeste123
```

Interceptação de tráfego

- Exemplo de comunicação sem criptografia
  - Imagine que um formulário envie esta requisição usando HTTP.

```
POST /login HTTP/1.1
Host: sistema.exemplo
Content-Type: application/x-www-form-urlencoded

usuario=aluno&senha=SenhaDeTeste123
```

- Quem conseguir capturar esse tráfego poderá ler o conteúdo do formulário.
- **Importante**: utilizar POST não criptografa os dados. Ele define como a requisição HTTP é organizada.

O que muda com HTTPS?

- Com HTTPS, o conteúdo HTTP é protegido pelo TLS durante o transporte. Um observador da rede, sem acesso às chaves ou aos dispositivos envolvidos, normalmente não consegue ler a senha, os cabeçalhos HTTP nem o corpo da requisição.
- Entretanto, algumas informações ainda podem ser observadas, como:
    - IPs envolvidos.
    - Portas.
    - Volume de dados.
    - Horários e duração das comunicações.
- Como reduzir o risco?
    - Utilizar HTTPS ou outro protocolo seguro apropriado.
    - Validar corretamente o certificado do servidor.
    - Não enviar dados sensíveis por protocolos sem proteção.
    - Controlar o acesso à infraestrutura de rede.

HTTP / HTTPS

- Perguntinha: Se uma senha é enviada no corpo de uma requisição POST usando HTTP, ela está protegida?

Ataque de intermediário: MITM

- O que é?
  - O ataque de intermediário, conhecido como Man-in-the-middle ou MITM, ocorre quando um atacante se coloca na comunicação entre duas partes e tenta observar ou manipular essa troca.
  - As partes podem acreditar que estão se comunicando diretamente, enquanto o atacante interfere no caminho.
  - O que o atacante pode tentar fazer?
    - Ler informações transmitidas sem proteção.
    - Modificar mensagens.
    - Substituir respostas.
    - Redirecionar a comunicação.
    - Se passar por um servidor legítimo.
    - Capturar credenciais ou tokens.

Ataque de intermediário: MITM

- **Exemplo didático**
    - Um usuário tenta acessar um sistema. Um intermediário tenta apresentar um servidor falso como se fosse o verdadeiro.
    - Se a aplicação não verificar corretamente a identidade do servidor, poderá enviar informações sensíveis para esse intermediário.

Diferença entre sniffing e MITM

Sniffing	MITM
Consiste em capturar e analisar tráfego	Envolve intermediar a comunicação
Pode ocorrer de forma passiva	Pode envolver manipulação ativa
Não exige, por si só, alterar mensagens	Pode modificar, substituir ou redirecionar mensagens

Por que os certificados são importantes?

- No HTTPS, o certificado participa da verificação da identidade do servidor.
- O cliente verifica, entre outros aspectos:
  - Se o certificado corresponde ao nome do servidor acessado.
  - Se está dentro do período de validade.
  - Se sua cadeia de certificação é confiável.
- Ter criptografia sem verificar com quem estamos falando não é suficiente: poderíamos estabelecer uma comunicação criptografada com o próprio atacante.
- Situações que aumentam o risco
  - Ignorar alertas de certificado.
  - Desativar a validação de certificados no código.
  - Aceitar qualquer certificado indiscriminadamente.
  - Utilizar protocolos sem autenticação e criptografia.
- Como proteger?
  - Utilizar TLS corretamente configurado.
  - Validar certificado e nome do servidor.
  - Corrigir problemas de certificado em vez de ignorá-los.
  - Manter sistemas e bibliotecas atualizados.

Conexão criptografada vs Servidor Falso

- Perguntinha: Se a conexão está criptografada, mas foi estabelecida com um servidor falso, estamos seguros?

Conexão criptografada vs Servidor Falso

- Perguntinha: Se a conexão está criptografada, mas foi estabelecida com um servidor falso, estamos seguros? **Não, também precisamos verificar a identidade do servidor.**

O que HTTPS não resolve?

- HTTPS não corrige automaticamente:
    - Falhas de autorização.
    - SQL injection.
    - XSS.
    - Senhas fracas.
    - Roubo de informações em dispositivos comprometidos.
    - Exposição de dados em logs.
    - Todos os tipos de ataque de negação de serviço.
- A proteção também termina no ponto em que o TLS é encerrado. Se um proxy recebe HTTPS e encaminha HTTP para outro servidor, esse segundo trecho precisa ser avaliado separadamente.

Segurança também está no código

- Uma conexão segura protege o transporte, mas a aplicação continua responsável por decidir quais dados aceitar, quem pode acessar cada recurso e como utilizar os recursos do servidor.
    - **Definir timeouts**
        - Timeout é um limite de tempo para determinada operação.
        - Sem limites adequados, uma aplicação pode ficar esperando indefinidamente por uma resposta, mantendo conexões e outros recursos ocupados.
        - Exemplo: uma API consulta um serviço externo. Se ele parar de responder, a aplicação deve interromper a espera após um período definido e tratar a falha.
    - Os limites devem considerar o tipo de operação. Uma consulta simples e um upload grande podem precisar de tempos diferentes.

Segurança também está no código

- **Limitar o tamanho dos dados**
    - A aplicação deve definir limites para:
        - Corpo de requisições.
        - Arquivos enviados.
        - Campos de texto.
        - Mensagens recebidas.
        - Quantidade de itens processados por operação.
    - Exemplo: se um comentário deve ter até 2.000 caracteres, o servidor precisa verificar esse limite. A validação apenas no navegador pode ser contornada.
    - Esses limites ajudam a reduzir consumo excessivo de memória, armazenamento e processamento.

Segurança também está no código

- **Tratar falhas e desconexões**
    - A rede pode apresentar:
        - Perda de conectividade.
        - Respostas atrasadas.
        - Interrupções.
        - Serviços indisponíveis.
    - A aplicação deve responder de forma controlada, registrar informações úteis e evitar estados inconsistentes.
    - Exemplo: se uma solicitação de pagamento ficar sem resposta, não significa necessariamente que o pagamento falhou. Repeti-la sem cuidado pode gerar duplicidade.
    - Quando necessário, utilizar operações idempotentes ou mecanismos de duplicação.

Segurança também está no código

- **Aplicar limites de requisições**
  - O *rate limiting* limita a quantidade de solicitações em determinado período.
  - Pode ajudar a reduzir:
    - Tentativas repetidas de login.
    - Abuso de funcionalidades.
    - Consumo excessivo de recursos.
    - Envios automatizados em grande volume.
  - Exemplo didático: permitir um número limitado de tentativas de login em uma janela de tempo, combinando critérios como conta e origem.
  - Limitar somente por IP pode afetar usuários legítimos que compartilham a mesma rede e não resolve todos os tipos de abuso.

Atividades

Atividades

1. TCP e UDP: confiabilidade e segurança

Explique as principais diferenças entre TCP e UDP. Por que o TCP confirmar o recebimento e retransmitir dados perdidos não significa que a comunicação está segura? Cite um mecanismo que pode proteger aplicações que utilizam TCP e outro para aplicações que utilizam UDP.

2. Os objetivos da segurança

Uma escola possui um sistema no qual os alunos consultam suas notas e os professores registram as avaliações. Explique como confidencialidade, integridade, autenticidade e disponibilidade se aplicam a esse sistema. Apresente um exemplo de problema relacionado a cada objetivo.

3. Interceptação de dados

Um desenvolvedor criou uma tela de login que envia usuário e senha pelo método POST, mas utiliza HTTP. Ele afirma que a senha está protegida porque não aparece na URL.

Essa afirmação está correta? Explique o que poderia acontecer se alguém capturasse esse tráfego e como a comunicação deveria ser protegida.

Atividades

4. Ataque de intermediário e certificados

Ao integrar uma aplicação a uma API, uma desenvolvedora encontrou um erro de certificado. Para fazer a integração funcionar, ela desativou a validação do certificado no código.

Explique por que essa decisão é perigosa, como ela pode facilitar um ataque de intermediário e qual seria a conduta adequada para resolver o problema.

5. Segurança no código

Uma API utiliza HTTPS, mas apresenta as seguintes características:

- Qualquer usuário autenticado consegue consultar dados de outros usuários alterando o identificador na URL.
- Não existe limite de tentativas de login.
- As senhas recebidas são registradas nos logs.
- Não há limite de tamanho para arquivos enviados.

Identifique o risco de cada uma das quatro situações e proponha uma correção para cada uma. Explique também por que o HTTPS não resolve esses problemas.

SENAI

Serviço Nacional de Aprendizagem Industrial

PELO FUTURO DO TRABALHO

0800 048 1212 **sc.senai.br**

Rodovia Admar Gonzaga, 2765 - Itacorubi - 88034-001 - Florianópolis, SC